

LOCALITY SENSITIVE HASHING

image is a long vector of pixel colours.

1	2	1
0	2	1
0	1	0

• add distance function $d(x_1, x_2)$

if distance is 0 $\rightarrow$ similarity is found

1 $\rightarrow$ no similarity.

★ time complexity

is n^2

• find near neighbours. $\rightarrow$ Jaccard distance/similarity.

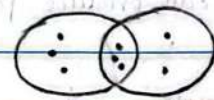
$$\text{sim}(C_1, C_2) = |C_1 \cap C_2| / |C_1 \cup C_2|$$

$$\text{Jaccard sim} = 3/8$$

$$\text{Jaccard distance} = d(C_1, C_2) = 1 - |C_1 \cap C_2| / |C_1 \cup C_2|$$

$$\text{Jaccard dist} = 5/8$$

3 STEPS FOR SIMILAR DOCS.



1. Shingling $\rightarrow$ convert documents to text/sets.

2. Min-hashing $\rightarrow$ convert large sets to short signatures

3. locality sensitive hashing $\rightarrow$ pair of signatures likely to be similar abc, bca, cab

★ hashing converts string to numerical

$$D_1 \text{ and } D_2 = 3/5 \rightarrow 1 - \frac{3}{5}$$

$$D_1 \text{ and } D_3 = 2/8 \rightarrow 1 - \frac{2}{8}$$

$$D_2 \text{ and } D_3 = 2/8 \rightarrow 1 - \frac{2}{8}$$

Document $\rightarrow$ Shingling $\rightarrow$ Minhashing $\rightarrow$ LSH $\rightarrow$

set of shingles
that appear
in the
document

short integers
that represent
the sets, and
reflect the
similarity.

pair of
signatures
to test for
similarity.

$K=5$ for short documents.

$K=10$ for long documents.

SHINGLING

• sequence of k characters that appears in the document.

• example $\rightarrow K=2$, doc = abcab, shingles $\rightarrow ab, bc, ca$

• k should be large enough, 5 for short, 10 for long.

• compress long shingles, hash them to 4 bytes (integer)

MINHASHING

• When is similarity interesting?

↳ When sets are large

• finding similar columns.

↳ compute signature of column = small summaries of columns!

↳ examine pair of signatures

• Warnings

↳ comparing all pairs of signature may take too much time.

↳ methods can produce false negatives, and false positive.

Minhashing example.

1	0	1	0
1	0	0	1
0	1	0	1
0	1	0	1
0	1	0	1
1	0	1	0
1	0	1	0

3	4
4	2
7	1
6	3
1	6
2	7
5	5

signature matrix M

2	1	2	1
2	1	4	1
1	2	1	2

first, fill the boxes with ∞ , then select row 1, and replace infinity with row number if number is smaller.

$M(i, c) \rightarrow c =$ number of columns in original matrix
 $i =$ number of columns in signature

Signature matrix $\times$ M

1 and 2 = 0 2 and 3 = 0

1 and 2 = 0 2 and 3 = 0

1 and 3 = $\frac{3}{4}$ 3 and 4 = 0

1 and 3 = $\frac{2}{3}$ 2 and 4 = $\frac{3}{3}$

1 and 4 = $\frac{1}{7}$ 2 and 4 = $\frac{3}{4}$

1 and 4 = 0 3 and 4 = 0

Similarities $\rightarrow$ 1-3 2-4 1-2 3-4

Col/Col 0.75 0.75 0 0

Sig/Sig 0.67 1.00 0 0

STEPS

• minhashing, generate signature matrix and find similarity of each column.

• select a threshold t ,

LOCALITY SENSITIVE HASHING

- finding similar pairs.
- checking all pairs is hard.
- use a function $f(x, y)$ to tell whether x and y is a candidate pair.
- solution $\rightarrow$ hash columns to many buckets.
- pick a similarity threshold s , fraction < 1
- $M(i, c) = M(i, d)$ for atleast fraction s values.

FP = dissimilar pair hashing to same bucket.

FN = similar pair never hashing to same bucket.

LSH FOR MINHASH SIGNATURES

- hash column for signature matrix M

PARTITION BANDS

PARTITION BANDS

- Divide matrix M into bands of r rows.
- For each band, hash its portion of each column to a hash table with k buckets, k should be large.
- Candidate column pairs hash to the same bucket

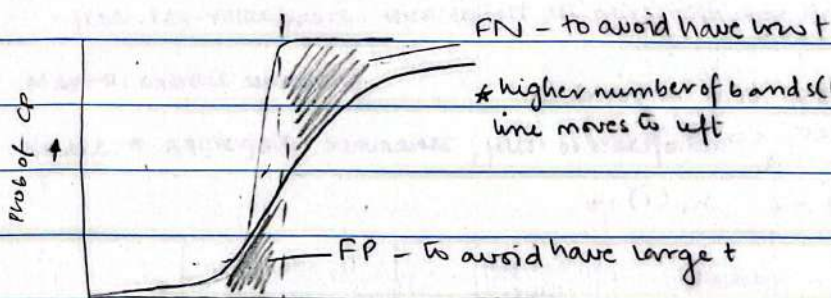
for > 1 band.

b bands

r rows per band

one band

Matrix M



$$s = 1 - (1 - s^r)^b$$

$$\text{threshold} = \left(\frac{1}{b}\right)^{1/r}$$

Jaccard $\uparrow$ similarity of documents $\rightarrow$ also known as S-curve.

- probability that the signatures agree in all rows of one particular band is s^r
- " " " signatures disagree in that atleast one row of a particular band is $1 - s^r$
- " " " " " " of each bands is $(1 - s^r)^b$
- " " " signatures agree in all the rows of atleast one band, and candidate pairs $1 - (1 - s^r)^b$

COMPUTING MINHASH SIGNATURES

Row	S ₁	S ₂	S ₃	S ₄	$x+1 \bmod 5(h_1)$	$3x+1 \bmod 5(h_2)$
0	1	0	0	1	1	1
1	0	0	1	0	2	4
2	0	1	0	1	3	2
3	1	0	1	1	4	0
4	0	0	1	0	0	3

S₁ S₂ S₃ S₄

→ firstly we initialize matrix

→ finding similarities

h₁ ∞ ∞ ∞ ∞

with ∞

original After

S₁-S₂

0

0

h₂ ∞ ∞ ∞ ∞

h

S₁-S₃

1/4

S₁-S₄

2/3

S₁ S₂ S₃ S₄

→ 1 is less than ∞, so we

S₂-S₃

0

0

h₁ 1 ∞ ∞ 1

replace S₁ and S₄ with

S₂-S₄

1/3

0

h₂ 1 ∞ ∞ 1

hash function.

h₁(0) = 1 h₂(0) = 1

S₃-S₄

1/5

S₁ S₂ S₃ S₄

→ 2 is less than ∞, and 4

as well, so S₃ is replaced.

h₁(1) = 2 h₂(1) = 4

h₁ 1 ∞ 2 1

h₂ 1 ∞ 4 1

S₁ S₂ S₃ S₄

→ h₁(2) = 3, h₂(2) = 2

S₂ can be replaced but not S₄ for h₁

S₂ can be replaced but not S₄ for h₂

h₁ 1 3 2 1

h₂ 1 2 4 1

S₁ S₂ S₃ S₄

→ h₁(3) = 4 h₂(3) = 0

no value is replaced in h₁

S₁, S₃, S₄ is replaced because

0 is less than (1, 4, 1).

h₁ 1 3 2 1

h₂ 0 2 0 0

HADOOP

- framework for distributed processing of large data sets.
- designed to detect and handle failures at application layer.
- large scale computing

idea: bring computation close to the data.

store files multiple times for reliability.

issue: copying data over a network takes time.

- MapReduce addresses this problem

↳ storage infrastructure

↳ programming model

- core components

↳ common, distributed file system, YARN, MapReduce

↳ namenode, stores metadata about where files are stored.

MapReduce → word counting.

- find key-value pairs, then collect all pairs with same Key.

ranking webpages
by popularity.

Map → extract something

Reduce → aggregate, summarize, filter or transform.

* master node takes care of
co-ordination

* master pings workers
periodically to detect failures.

	(The, 1)	(crew, 1)	(crew, 2)
	(crew, 1)	(space, 1)	(space, 1)
	(of, 1)	(the, 1)	(the, 1)
	(the, 1)	(the, 1)	
	(space, 1)	(the, 1)	

Big
Document

↓
map
read input and
produces set of
key-value pairs

↓
group by
key, collect
all pairs
with same
key

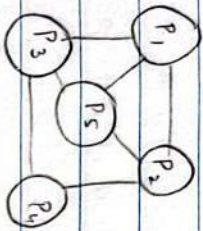
↓
reduce
collect all
values belonging
to key and output

PAGE RANK

The value of $r(P_i)$ is the sum of normalized pageranks of all webpages pointing into P_i .

$$r(P_i) = \sum r(P_j) - \text{rank of previous iteration}$$

$|P_j|$ - outgoing links.



Pages	P ₁	P ₂	P ₃	P ₄	P ₅
PageRank	3.5	1.2	1.2	1.2	1.2

* Incoming links to P₅

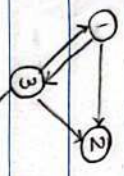


P in the beginning, all pageranks are unknown. How to determine first pagerank value?

• give initial pagerank to every webpage, assuming probability = $\frac{1}{\text{no. of nodes}}$

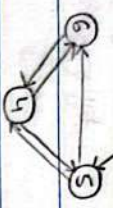
• perform calculation of pagerank iteratively until change is negligible

Example:



$$r_0(P_i) = \frac{1}{6} \text{ for } i = 1, 2, \dots, 6$$

Iteration 1:



$$r_1(P_1) = r_3(P_2) \cdot \frac{1}{3} = 0.0556$$

$$r_2(P_2) = r_1(P_1) \cdot \frac{1}{2} + r_3(P_3) \cdot \frac{1}{3} = 0.1389$$

$$r_3(P_3) = r_2(P_1) \cdot \frac{1}{2} = 0.0833$$

$$r_4(P_4) = r_3(P_2) \cdot \frac{1}{2} + r_5(P_5) = 0.25$$

$$r_1(P_5) = r_4(P_4) \cdot \frac{1}{2} + r_5(P_5) \cdot \frac{1}{2}$$

$$\pi^{(0)} = \left(\frac{1}{6} \quad \frac{1}{6} \quad \frac{1}{6} \quad \frac{1}{6} \quad \frac{1}{6} \quad \frac{1}{6} \right)$$

$$H = \begin{pmatrix} 0 & 1/2 & 1/2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 1/3 & 1/3 & 0 & 0 & 1/3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1/2 \\ 0 & 0 & 0 & 0 & 1/2 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1/2 \end{pmatrix}$$

$$\pi^{(1)} = \pi^{(0)} H$$

rows of matrix H with all zeros

are replaced by rows of $\left(\frac{1}{n} \quad \frac{1}{n} \quad \dots \quad \frac{1}{n} \right)$

MATRIX AND VECTOR NOTATIONS

row vector $1 \times m$

column vector $n \times 1$

PAGERANK VECTOR

$\pi = (r(p_1) \ r(p_2) \ \dots \ r(p_n))$ $\therefore$ let π^k denote vector at iteration k

at iteration 0 $\rightarrow \pi^{(0)} = (\frac{1}{n} \ \frac{1}{n} \ \dots \ \frac{1}{n})$

DANGLING PAGES

• These pages have no outgoing link as random surfer cannot proceed forward from these pages.

e.g. pdf files, image, audio

H' is a stochastic matrix as all rows equal to 1

$G = \alpha H' + (1-\alpha)(\frac{1}{n})I$ let $\alpha = 0.9$

$$G = 0.9 \begin{pmatrix} 0 & \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 \\ \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} \\ \frac{1}{3} & \frac{1}{3} & 0 & 0 & \frac{1}{3} & 0 \\ 0 & 0 & 0 & 0 & \frac{1}{2} & \frac{1}{2} \\ 0 & 0 & 0 & \frac{1}{2} & 0 & \frac{1}{2} \\ 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix} + (0.1) \left(\frac{1}{6} \right) \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \end{pmatrix}$$

1's matrix

RANK SINKS

• those pages that accumulate more and more pagerank at each iteration.

* The pages having no outgoing links are called Dangling Pages.

↳ pdf, image, data, pages with no hyperlink

ADJUSTMENT (TELEPORTING)

• The random surfer after entering a dangling node, can now hyperlink to any page at random.

• multiplying our stochastic matrix H' with π^k

• we get a probability vector $\rightarrow$ sum of values of vector (π^k is 1)

↳ no pagerank is lost.

• allow teleporting to any page at any time.

$\alpha \rightarrow$ random surfer will follow one of hyperlinks at current page. — continues to click on page links

$1 - \alpha \rightarrow$ random surfer will randomly select a page out of n pages.

$\rightarrow$ The modified hyperlink matrix is called Google matrix;

$$G = \alpha H' + (1 - \alpha) \left(\frac{1}{n} \right) I$$

$I \rightarrow n \times n$ matrix with all entries

back page: $G = \alpha H' + (1 - \alpha) \left(\frac{1}{n} \right) I$

↳ surfer can jump to any page at random with probability $(1 - \alpha)$
• 1's matrix

equal to 1

$$\alpha = 0.85$$

* $\pi^{(k+1)} = \pi^k G \rightarrow$ the pagerank vector converges to a stable value.

α should be relatively large to reduce the effect of random teleporting.

$1 \rightarrow$ average number of steps before jumping to random page.

$$1 - \alpha$$

EXAM scenario $\rightarrow$ transaction, connectivity matrix or extract connectivity matrix from description

band size should be reasonable,

$$= \frac{\text{size of minhash}}{\text{row} \times \text{column}} * 4$$

band 1 {

2	1	2	
1	2	1	

band 2 {

— same bucket, but if band size is 1, more false positives

if band size is large, more false negative.

CONTINUATION LOCALITY SENSITIVE HASHING

NUMBER BANDS

- increasing no. of bands, then false positive goes up. as candidate pairs increase.
- having only 1 row/band which means all rows of minhash matrix in a single band, FN increases.

INCREASING NUMBER OF BUCKETS

- False Positive $\rightarrow$ decreases, as it reduces the likelihood of dissimilar items being hashed to same bucket.

False Negative $\rightarrow$ increases, similar items might hash to different bucket if their hash values lie on bucket boundaries.

DECREASING NUMBER OF BUCKETS

- False Positive $\rightarrow$ increase, higher likelihood of dissimilar item being hashed to same bucket
- False Negative $\rightarrow$ decrease, similar items likely to hash in same bucket, reduces chances of missing out on genuine similarities.

INCREASING NUMBER OF HASH FUNCTIONS

- False Positive $\rightarrow$ decreases, less likely for dissimilar items to be considered similar
- False Negative $\rightarrow$ increases, genuinely similar items being missed as they fail to match one or more additional hash function.

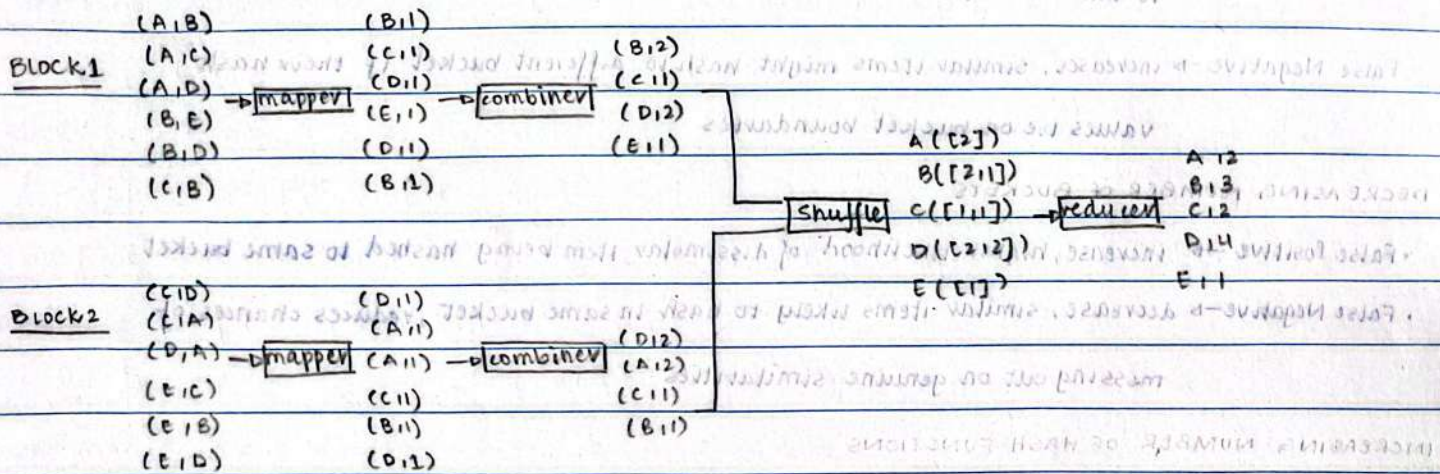
DECREASING NUMBER OF HASH FUNCTION

- False Positive $\rightarrow$ increases, increasing likelihood of dissimilar items being hashed to same bucket.
- False Negative $\rightarrow$ decreases, genuinely similar items to be considered as such, reduces chances of missing similarities.

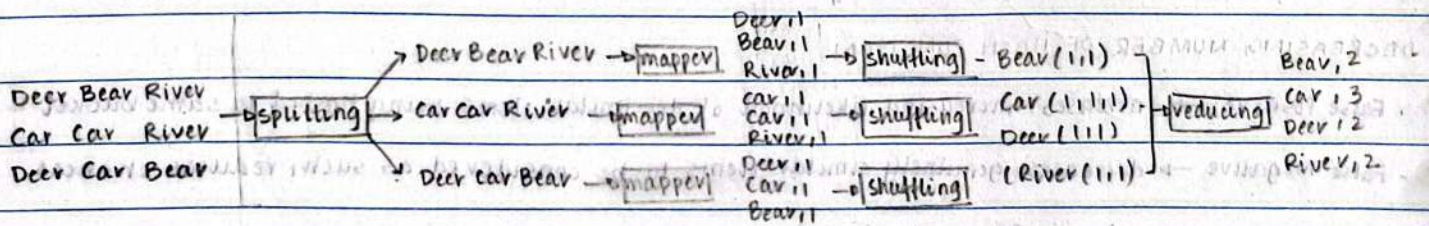
RULE OF THUMB

- make M much larger than the number of nodes in the cluster.
- because output is spread across R files. so R is smaller than M .

- combiner works only if reduce function is commutative and associative
- much less data needs to be copied and shuffled.



mapreduce word count process that needs more shuffling.



FREQUENT ITEMSETS

• support for itemset I : number of basket containing all items in I .

Items = {milk, coke, pepsi, banana, juice}

support threshold = 3

$B_1 = \{m, c, b\}$ $B_3 = \{m, b\}$ $B_5 = \{m, p, b\}$ $B_7 = \{c, b, j\}$

$B_2 = \{m, p, j\}$ $B_4 = \{c, j\}$ $B_6 = \{m, c, b, j\}$ $B_8 = \{b, c\}$

• frequent itemsets : {m}, {c}, {b}, {j}

ASSOCIATION RULES

$I \rightarrow j$ where I is a set of item and j is an item.

∴ if all the items in I appear in some basket, then j is likely to appear in that basket as well.

$$\text{conf}(I \rightarrow j) = \frac{\text{support}(I \cup j)}{\text{support}(I)}$$

∴ example $\rightarrow \{m, b\} \rightarrow c = 2/4 = 0.5$

• Interest of an association rule $I \rightarrow j$:

$$\text{Interest}(I \rightarrow j) = \text{conf}(I \rightarrow j) - \text{Pr}[j] \quad \text{--- } j / \text{total baskets.}$$

∴ interesting rules are those with high interest values (above 0.5)

∴ if confidence is high, then interest cannot be negative.

• for every subset A of I , a rule $A \rightarrow I/A$, since I is frequent A is also.

if $A, B, C \rightarrow D$ is below confidence, so is $A, B \rightarrow C, D$

• to reduce number of rules, we can post process them.

- maximal frequent itemset

- closed itemset.

A - PRIORI ALGORITHM

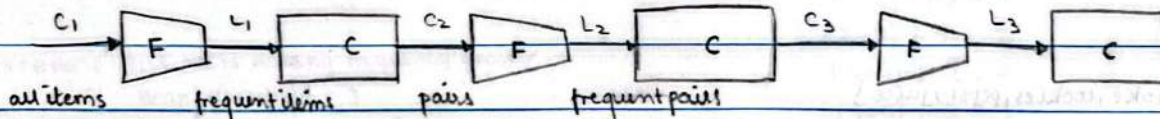
- two pass approach limits need for main memory to find pairs.
- if a set of items I appears atleast s times, so does every subset J of I
- if item (i) does not appear in s baskets, then no pair including (i) can appear in s baskets.

Database (Support = 2)

Tid	Items	1 st scan	Itemset	sup	2 nd scan	Itemset	sup	3 rd scan	Itemset	sup
10	A, C, D		A	2		A, B	1		A, B, C	1
20	B, C, E		B	3		A, C	2		A, B, E	1
30	A, B, C, E		C	3		A, E	1		A, C, E	1
40	B, E		D	1		B, C	2		B, C, E	2
			E	3		B, E	3			
						C, E	2			

Item names to integers	1 2 n	Item counts
unused		

Item names to integers	1 2 n	Item counts
count pairs.		



PARK - CHEN - YU ALGORITHM

DATE: / /

Itemname to integer	Item counts	Itemname to integer	frequent items
		bitmap	hash table contains count so bigger
hash table for bucket counts	as large as possible	count pairs	

- for each bucket keep the count, not the actual pairs that hash.
- for a bucket with total count less than s , none of its pairs can be frequent.
- pass 1 $\rightarrow$ maintain a hash table
- pass 2 $\rightarrow$ only count pairs $\cdot$ both i and j are frequent $\cdot$ the pair hashes to the bucket whose bit in the bit vector is 1.
- if the count of a bucket is at least as great as the support thresholds, it is a frequent bucket.
- bitmap $\rightarrow$ 1 if bucket is frequent, 0 if not.
- $\rightarrow$ 4 byte integer count is replaced by bits.

- the hashtable must eliminate approx $2/3$ of the candidate pairs for PCY to beat A-Priori
- cannot use triangular matrix because pairs of frequent items that PCY lets us avoid counting are placed randomly within the triangular matrix; pairs that hash to infrequent buckets

example

Items = {milk, coke, cookies, pepsi, juice}

$B_1 = \{m, c, b\}$	$B_3 = \{m, b\}$	$B_5 = \{m, p, b\}$	$B_7 = \{c, b, j\}$	$s = 3$
$B_2 = \{m, p, j\}$	$B_4 = \{c, j\}$	$B_6 = \{m, c, b, j\}$	$B_8 = \{b, c\}$	milk = 1, coke = 2, cookies = 3, pepsi = 4, juice = 5
$B_1 = \{1, 2, 3\}$	$B_3 = \{1, 3\}$	$B_5 = \{1, 3, 4\}$	$B_7 = \{2, 3, 5\}$	
$B_2 = \{1, 4, 5\}$	$B_4 = \{2, 5\}$	$B_6 = \{1, 2, 3, 5\}$	$B_8 = \{2, 3\}$	

hash function = $(i+j) \% 5 = k$

Pass 1	Item#	Count	HashTable	Bucket #	Count
	1	5		0	
	2	5		1	
	3	5		2	
	4	2		3	
	5	4		4	

STEP#1

item# count

1	5
2	5
3	6
4	2
5	4

* bit vector: frequency of candidate pair is
greater than equal to the threshold
then bit vector is 1

bitvector bucketno count pairs

1 0 4 (2,3)

1 2 3 (2,5)

1 4 4 (1,3)

STEP#2: remove all item having
count less than 1 so

candidate set = {1,2,3,4,5}

STEP#3: map all candidate pairs, and
calculate frequency, avoid
duplicates.

B1: (1,2) (1,3) (2,3) → 2, 4, 4

B2: (1,4) (1,5) (4,5) → 2, 2, 1

B3: (-)

B4: (2,5) → 3

B5: (3,4) → 1

B6: (3,5) → 2

B7: (-)

B8: (-)

PASS 1.

Bucket# count

(1+2) * 0.5 = 3

0 6

(1+3) * 0.5 = 4

1 2

(2+3) * 0.5 = 0

2 4

(1+4) * 0.5 = 0

3 4

(1+5) * 0.5 = 1

4 5

(4+5) * 0.5 = 4

(2+5) * 0.5 = 2

(3+4) * 0.5 = 2

(3+5) * 0.5 = 3

STEP#4: list pairs having frequency greater
than threshold = 3

= (1,3) (2,3) (2,5)

STEP#5: hash function → (i+j) * 0.5 = k

(1,3) = (1+3) * 0.5 = 4

(2,3) = (2+3) * 0.5 = 0

(2,5) = (2+5) * 0.5 = 2

FILTERING DATA STREAMS

Bloom filter $\rightarrow$ keeps list of all the URL it has found so far.

• it needs to filter out those URL's it has seen before.

ROLE OF BLOOM FILTER

- certain URL have been seen before
- others will be declared new, and added to the list of URL
- it has false positive meaning it can declare a URL has been seen before when it hasn't
- it has no false negative meaning if it says never seen, then it is true

WORKING

- array of bits, together with hash functions.
- argument of each hash function is a stream element, and returns a position
- initially, bits = 0
- we set 1 to the bits $h(x)$, for each hash function.

example:

$N = 11$ bits. stream elements = integers.

$h_1(x)$ = odd-numbered bits in binary then take mod 11

$h_2(x)$ = even-numbered bits in binary then take mod 11

stream element	$h_1(x)$	$h_2(x)$	filter contents
undelined = odd.			00000000000
25 = 11001	5	2	$\begin{array}{ccccccccccc} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{array}$
159 = 10011111	7	0	101001010000
585 = 1001001001	9	7	101001010100

101 (odd) = $5 \% 11 = 5$
 10 (even) = $2 \% 11 = 2$
 0111 (odd) = $7 \% 11 = 7$
 1011 (even) = $11 \% 11 = 0$
 01001 (odd) = $9 \% 11 = 9$
 10010 (even) = $10 \% 11 = 10$

- suppose element y appears in the stream, we want to know if we have seen before.
- $compute_n(y)$
- if all resulting position are 1, we say we have seen y before.
- if atleast one position is 0, we have not seen y before. hence no false negative.

lookup example:

filter = 101001010100

lookup element $y = 118 = 1110110$

$h_1(y) = 14 \div 11 = 3$ odd

1110 (odd) $= 14 \div 11 = 3$

101 (even) $= 5 \div 11 = 5$

$h_2(y) = 5 \div 11 = 5$ even

→ bit 5 is 1, bit 3 is 0, we are sure $y = 118$ is not in the set.

PERFORMANCE OF BLOOM FILTER

- probability of false positive depends on 1's in the array. & number of hash functions.
- $= (\text{fraction of 1's})^{\# \text{ of hash function.}}$

EXAMPLE - THROWING DARTS.

- turning random bits from 0 to 1 is throwing d darts at t targets. m darts at n targets.
- targets = bits/buckets
- darts = hash values of items.
- probability of hitting any one of the target $(1/t)$
- probability of not hitting target $(1 - 1/t)$
- probability of not hitting target by all d darts $(1 - 1/t)^d$

∴ probability of being
false positive for any
un-seen element.

rewrite as $\rightarrow (1 - 1/t)^{t(d/t)} = e^{-d/t}$

example: we use array of 1 billion bits, 5 hash functions, insert 100 million elements.

$t = 10^9$, $d = 5 \times 10^8$, fraction of 0's $= e^{-1/2}$, fraction of 1's $= 0.393 (1 - e^{-d/t})$

probability of false positive $= (0.393)^5 = 0.00937$

$K = \frac{n}{m} \ln(2)$

$K = \frac{t}{d} \ln(2)$

FREQUENT ITEMSETS IN ≤ 2 PASSES.

1. Random sampling

2. SON algorithm.

random sampling:

FN: an itemset is frequent in whole but not in sample.

FP: an itemset frequent in sample but not in whole.

SON ALGORITHM

- repeatedly read small subsets of baskets

- an itemset becomes a candidate if it is found to be frequent in any one or more subsets of the baskets.

PASS 1 - map task.

- divide data into chunks.
- for each chunk $\text{support } cs = s / \text{number of chunks}$.
- find frequent itemset.

PASS 1 - reduce task

- Key part (F, 1) assigned to each reduce.
- produces keys that appear one or more times; these are candidate itemsets.

PASS 2 - map task

- takes output from first reduce task, and input data.
- map task will have candidate itemsets.
- counts occurrence of each candidate itemset
- output of map task (c, v)
 - c = candidate itemset
 - v = support of itemset among baskets that were input to map task.

PASS 2 - reduce task

- takes itemset, calculates sum of associated values for each itemset.
- support greater than or equal

SDN ALGORITHM EXAMPLE $s=3$

$$B_1 = \{m, c, b\} \quad B_2 = \{m, p, j\} \quad B_3 = \{m, c, b, n\} \quad B_4 = \{c, j\}$$

$$B_5 = \{m, p, b\} \quad B_6 = \{m, c, b, j\} \quad B_7 = \{c, b, j\} \quad B_8 = \{b, c\}$$

• chunk 1 = B_1, B_2, B_3, B_4

• chunk 2 = B_5, B_6, B_7, B_8

• reduced threshold

$$\text{support} = s / \text{number of chunks} = 3/2 = 1.5 = 2$$

1. chunk 1 - map 1

chunk 2 - map 2

output of reducer task

$m = 3$
 $c = 2$
 $b = 2$
 $p = 1$
 $j = 2$
 $m, c = 1$
 $m, b = 2$
 $c, b = 1$
 $m, p = 1$
 $p, j = 1$
 $m, j = 1$
 $c, j = 1$

$m = 2$
 $p = 1$
 $b = 4$
 $c = 3$
 $j = 2$
 $m, p = 1$
 $m, b = 2$
 $p, b = 1$
 $m, c = 1$
 $m, j = 1$
 $c, j = 2$
 $b, j = 3$
 $c, b = 3$

→ key 1

m
 c
 b
 j
 m, b
 c, j
 b, j
 c, b

2. chunk 1 - map 1

chunk 2 - map 2

output of reducer task.

$m = 3$
 $c = 2$
 $b = 2$
 $j = 2$
 $m, b = 2$
 $c, j = 1$
 $b, j = 0$
 $c, b = 1$

$m = 2$
 $c = 3$
 $b = 4$
 $j = 2$
 $m, b = 2$
 $c, j = 2$
 $b, j = 2$
 $c, b = 3$

→ same output

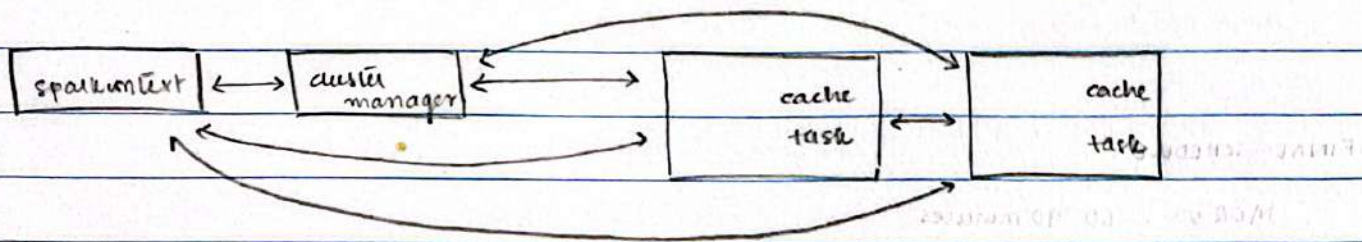
$m = 5$
 $c = 5$
 $b = 6$
 $j = 4$
 $m, b = 4$
 $c, j = 3$
 $b, j = 2$
 $c, b = 4$

APACHE SPARK

- is a fast and general engine for large-scale data.

hadoop performs batch processing, apache spark can do real time processing.

- spark is 10 times faster
- supports data analysis, ML, graphs
- spark driver is node that allows application to create SparkContext



- when master is called 'Driver' and 'slaves' are called Workers.
- resources managed by cluster manager.
- sparkContext available as sc

>>> sc = appName

RDD (RESILIENT DISTRIBUTED DATASET)

- R - if data in memory is lost, it can be recreated.
- D - processed across the cluster.
- RDD's are fundamental units of data - serve as data containers.
- in memory computation, results stored in RAM instead of disk.
- lazy evaluation, execution does not start until an action is triggered.
 → combines multiple transformations into efficient execution pipelines
- fault tolerant, track data lineage to reconstruct lost data automatically.
- immutability, created data can be retrieved anytime but its value can't be changed.
- partitioning, as they are large so cannot fit into a single node and must be partitioned across various nodes.
- persistence, optimization technique to save result of RDD. stores intermediate result.

- **transformations**: define a new RDD based on current ones.

- **actions**: return values.

RDD OPERATIONS.

- transformations create new RDD from an existing one.

- RDD's are **immutable**, i.e. they never change.

- transform in sequence to modify data as needed.

- transformations:

↳ **map()** → creates a new RDD by performing a function on each record.

↳ **filter()** → creates a new RDD by including or excluding each record

in the base RDD according to a boolean function.

- operations (actions)

↳ **count()** → number of elements.

↳ **take(n)** → array of first ^{rows} n elements.

↳ **collect()** → array of all elements.

↳ **saveAsTextFile(file)**, **show(n)** → display first n rows (default=20)

LAZY EXECUTION

- data in RDD is not processed until an action is performed.

file

my name is Tabidoh

↓
`val mydata = sc.textFile("file.txt")`

`val mydata2 = mydata.map(line → line.toUpperCase())`

MY NAME IS TABIDOH

`val mydata3 = mydata2.filter(line → line.startsWith("I"))`

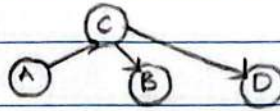
MY NAME

`mydata3.count()` → 1

- RDD's are organized in the form of Directed Acyclic Graphs.

→ nodes are RDD

→ arrows are transformation.



- pipelining → performs sequence of transformation by row

SPARK PERFORMANCE TUNING

- transformations are embedded in a chain but only evaluated when an action is called.
- persisting an RDD saves the data by default in memory.

`mydata2.persist()` ** allows to specify storage level at which RDD should be persisted.*

→ memory only

→ memory and disk: memory first, then spill to disk

→ disk only

- after calling `count`, which executes lineage, and persists RDD2 at the same time.
- the data is cleared from memory except the cached RDD that is stored in memory!

• `rdd.cache()`

- cache an RDD into memory.

↳ `rdd = sc.textFile("file.txt")`

↳ `rdd.count()`

↳ `rdd.cache()`

↳ `rdd.collect()`

- when cache is called, `count()` causes RDD to keep values it has calculated in memory.

- rdd uses cached values for calculating `collect()`.

- when not to use cache.

↳ depending on how much time dataset is accessed, re-computation can be faster than the price paid by the increased memory pressure.

↳ reading dataset only once, no point of caching. as work will be slow

PERSISTENCE LEVELS.

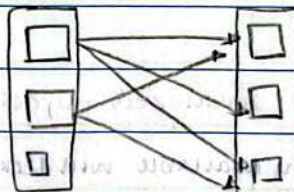
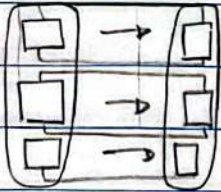
- memory only : store rdd , but some rdd wont fit , so partitions wont be cached.
- memory and disk . store rdd , if rdd doesn't fit in memory , store partitions that don't fit on disk and read them from disk.
- memory only ser . store rdd as serialized , more space-efficient , more cpu-intensive.
- memory and disk ser . store rdd as serialized , if RDD doesn't fit in memory , some partitions are evicted from memory and written to disk.
- disk only : store rdd partitions on disk rather than in memory . useful if RDD are much larger than available memory and reading from source will be time consuming.
- when spark serializes objects , it converts it into binary representation that can be efficiently stored and transmitted . takes up less space , so reduced memory usage.
- process involves encoding objects into sequence of bytes .
- spark uses java default serialization
- kryo : faster serialization and smaller serialized object .
- unpersisting : as we cache more RDD , memory available will decrease .

PISPAK

- spark df supports parallelization, pandas donot
- has multiple nodes, pandas has single nodes
- follows lazy execution, panda follows eager execution
- is immutable, pandas df is mutable
- is distributed, pandas df is not distributed hence slower.

NARROW TRANSFORMATIONS

- Map(): return a new rdd formed by passing of each element.
- Filter(): return a new dataset formed by selecting those elements that returns true.
- flatMap(): each input can be mapped to 0 or more output
- map returns a collection, flatmap returns flattened results.
- narrow → convert each input to only one output, is faster, doesnot require any data shuffling.
- wide → input to many output, is slower, require data shuffling over cluster.



map, flatmap, union,
sample, filter

intersection, distinct,
join, cartesian, coalesce.

- groupByKey ([numPartitions]) : returns dataset of (k, iterable <v>) pairs.
- reduceByKey (func, [numPartitions]) : returns dataset of (k,v) pairs where values for each key aggregated
- reduceByKey does a map side combine.

Task 1 → GREEN 1

RED, 1 RED 3

GREEN, 1

groupbykey

* transferred all elements over the network.

* could possibly result in an out of memory exception.

reducebykey - similar to combine(C).

Task 2 → RED, 1

GREEN 1 * sends fewer elements

RED, 1 GREEN, 1 → RED 3

RED, 1 RED, 2

• commutative : $(a+b)$

• associative : $(a+b)+c$

• distributive : $a \times (b+c)$

• creating dataframes

`from pyspark.sql import SQLContext`

`sqlctx = SQLContext(sc)`

`df = spark.read.load("path/to/csv.csv", format="csv", sep=",")`

`df = pd.read_csv("path/to/csv.csv")`

• schema : returns object describing data.

• printschema : displays schema as a visual tree.

• columns : returns array of name of columns.

• dtypes : returns an array of (column name, type) pairs.

• explain : prints debug information about dataframe.

QUERIES.

distinct : returns new df with distinct elements.

join : joins 2 df.

limit : a new df with first n rows.

select : a new df with data from one or more columns.

SQL VS NOSQL

SQL DATABASE

- data manipulation language : insert, update, select, delete.
- data definition language : create table, indexes, views, alter drop.
- create view : defines a virtual table based on results of a query.

ACID PROPERTIES

1. atomicity

→ all the changes are performed in its entirety or nothing at all.

2. consistency

→ data in consistent state when it starts and ends.

3. isolation

→ if multiple transactions are running separately / simultaneously, it should not be interrupted by one of those transaction.

4. durability

→ changes to the data persist and are not undone. even if system failure occurs.

database : oracle RDBMS, MySQL

FEATURES OF NOSQL DATABASE

Large data volumes : massive volume of structured and unstructured data.

scalable replication : ensures data availability and fault tolerance by maintaining multiple copies of data across distributed environment.

distributed database : NOSQL database are inherently distributed, operate across multiple nodes.

queries answered quickly.

mostly queries.

schema-less

simpler and faster.

BASE PROPERTIES NOSDL

1. Basically Available.

- prioritize high availability of data over immediate consistency.
- data is replicated across multiple nodes to ensure that it remains accessible even if individual nodes fail.
- provides uninterrupted access to data, even if node fails.

2. Soft state

- due to lack of immediate consistency, data values may change over time.
- different replicas don't have to be mutually consistent.

3. Eventually consistent

- strive to achieve consistency over time.
- data replicas may be inconsistent after an update.
- all replicas converge to a consistent state.

marketing and customer service companies who deal with sentiment analysis when conducting social network research use BASE - contains huge amount of data.

e.g. MongoDB

- ACID-compliant database will be a better fit for those who require consistency, predictability, and reliability.
- those who consider growth to be among their priorities will likely choose BASE as it enables easier scaling and more flexibility.

TYPES OF NOSQL

1. Document: each document contains pairs of field and value. → mongoDB
2. key-value: contains keys and values → redis.
3. wide-column stores: store in tables, rows + columns, allows each row to have different set of column
→ cassandra, Hbase
4. graph: store data in nodes and edges. nodes store information about people, place and edges describe relationship → neo4j, gremlin.

ROBMS VS NOSQL

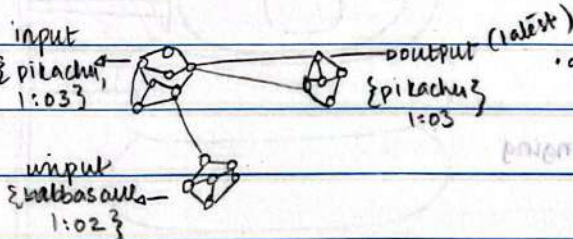
- in ROBMS, to store a user and its hobbies information we need two table whereas to save in mongodb one document is required.

CAP THEOREM

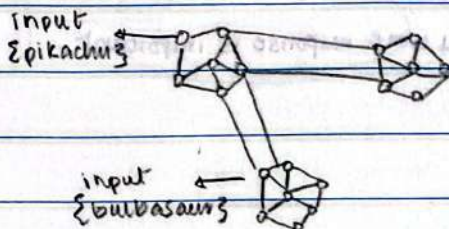
1. consistency: every read receives most recent write or an error.
2. availability: every request receives a response, but most recent write not guaranteed.
3. partition tolerance: system continues to operate despite network partitions between nodes.



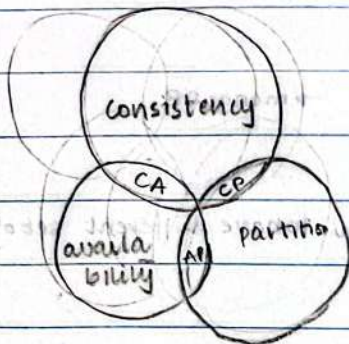
• doesn't result in failure of entire network, data records are replicated across combination of nodes.



• consistency: all nodes see the same data, performing read operation will return value of most recent write causing all nodes to return the same data. system can shift to inconsistency but gets rolled back if there is any error.



availability: system remains operational 100% of the time, every client gets a response, high availability isn't feasible when analyzing streaming data



RDBMS:

→ great at consistency.

→ okay at availability.

→ not great at partitioning.

CONSISTENCY AND AVAILABILITY

- these both are prioritized, partition tolerance may be sacrificed.
- ensuring that all nodes in the distributed system return the same value for a given data item.
- availability maintained, system continues to operate and serve requests even if nodes some fail.
- if network partition occurs, system may choose to sacrifice partition tolerance and become unavailable until partition is resolved.

CONSISTENCY AND PARTITION TOLERANCE

- availability sacrificed
- ensuring that all nodes agree on current state of data even in presence of network partitions.
- availability may be compromised as system becomes unavailable or experience downtime.

AVAILABILITY AND PARTITION TOLERANCE

- consistency sacrificed.
- systems prioritize uninterrupted access to data and services.
- achieving strong consistency across all nodes may be challenging

USECASES AND REQUIREMENT

- prioritize consistency over availability, such as financial systems → data integrity is critical.
- prioritize availability over consistency, such as social media where real time response is important.

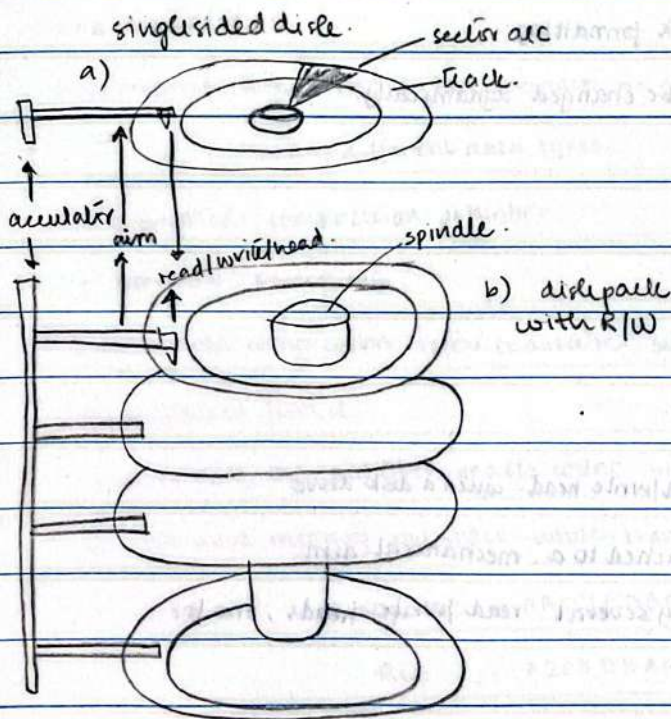
TYPES OF STORAGE MEDIA

- primary : CPU main memory, cache
- secondary : magnetic disks, solid state drives.
- tertiary : removable media.
- cache memory : static RAM
- mass storage : magnetic disks.

databases - typically stored on this

STORAGE ORGANIZATION OF DATABASE

- persistent data : information that is meant to be stored beyond the lifespan of a programs execution.
- transient data : managed using data structures and memory management techniques.
- file organization : how records are physically placed on disk and accessed.



* a sector is a small, fixed size unit of storage on a disk, it represents the smallest addressable unit of data on a disk surface.

EXPLAINING DISC- TRACKS .

- a disk is single sided if it stores information on one of its surface only.
- to increase storage capacity, disks are assembled into a disk pack.
- information stored on disk surface in concentric circles of small width.
- each circle is called a **track**.
- in disk packs, tracks with same diameter on the various surfaces are called a cylinder. hence data stored on one cylinder can be retrieved much faster than if distributed among different cylinders.
- capacity of track $\rightarrow$ 10's kilobytes to 150 kbytes
- **track** divided into **smaller sector** as it contains large amount of information and cannot be changed.
- **blocks** are larger units of data storage that consist of multiple sectors grouped together.
- division of track set by operating system during disk formatting.
- block size fixed during initialization and cannot be changed dynamically.

block size range from 512 to 8192 bytes

DATA ACCESS

- a disk is a random access addressable device.
- **hardware address of a block is a combination of a cylinder number + track number + block number**
- mechanism that reads or writes a block is the disk read/write head. called a disk drive.
- read/write head includes an electronic component attached to a mechanical arm.
- disk pack with multiple surface are controlled by several read/write heads, one for each surface.
- all arms are connected to an actuator attached to another electrical motor which moves read/write heads.
- **storage space $\rightarrow$ #surfaces * #tracks * #blocks * size of each block**

Disc access time $\rightarrow$ time = seektime + rotational delay + transfer time + other.

• seek time: time required to position the read/write head on correct track is called seek time.

• typical seek times are 5 to 10 msec on desktops and 3 to 8 msec on servers.

• rotational delay: $R = \frac{1}{2}$ revolution read/write head can access data from sides of the disk.

• transfer time: transfer several consecutive blocks on same track or cylinder. $\rightarrow$ time to actually read or write data since the read/write head is in position

• other delays: CPU time to issue I/O

PROS AND CONS.

Row based $\rightarrow$ compression, aggregation not efficient, horizontal partitioning, OLTP, optimal for read, write.

column based $\rightarrow$ writes are slower, OLAP, vertical partitioning, inefficient queries when accessing multiple columns.

DATA COMPRESSION

Row oriented: compression is low because we don't have homogenous data in each block means different data types.

column oriented: compression is higher

RUN LENGTH ENCODING

• lossless data compression refers to data in such a way that the original form of the data can be derived from it.

• sent it with the character and its count instead of sending it multiple times.

• we can save memory and space while transmitting data.

example: sequence : AACCCBBBBBAAAAFFFFFFF

RLE : A2C3B5A4F8

* exclamation mark used as a separator between character and its count, helps in decompressing data